

1.1

Compare cumulative cost during training (plots):

The PredictiveAgent accumulates cost at a dramatically higher rate than both RL agents, reaching approximately 1.35 million by episode 5000, compared to ~640,000 for both Q-Learning and SARSA. This is expected — the PredictiveAgent uses a fixed, handcrafted policy with no learning, so it never improves. Its perfectly linear training curve confirms it behaves consistently but poorly, incurring roughly the same cost every episode with no adaptation.

Both Q-Learning and SARSA show a slight deceleration in their training curves over time, which is a sign that they are gradually improving their policy as learning progresses.

The PredictiveAgent starts with the highest win rate (~9-10% around episode 100) but then *declines* steadily to ~6.8% by episode 5000, suggesting it never truly learns — it relies on a fixed heuristic rather than updating from experience, so accumulated cost doesn't improve. Q-Learning and SARSA both start near 0% but improve continuously throughout training, with Q-Learning pulling slightly ahead of SARSA in the early-to-mid episodes (it reaches ~18% at ep. 2000 vs SARSA's ~22%, though SARSA actually converges faster initially around ep. 600-1000). By episode 5000 both land nearly identically at ~29.2-29.3% during training, meaning their cumulative training costs are comparable, with Q-Learning accumulating somewhat more cost early on due to slower initial learning.

Test-time performance (cumulative costs, win-rate, avg cost, avg steps):

The contrast is even clearer at test time. The PredictiveAgent's cumulative cost grows almost perfectly linearly to ~135,000 over 500 episodes, meaning the time outside the bar always outweighs the time inside.

In contrast, both Q-Learning and SARSA show negative cumulative cost regions, especially in the early-to-mid test episodes, indicating they successfully keep the fish inside the bar at times. This demonstrates that the RL agents genuinely learned something useful from interaction, while the PredictiveAgent did not.

At test time the gap becomes clearer: Q-Learning achieves 36.8% win rate at 85.1 avg cost, SARSA gets 33.0% at 103.1 avg cost, and PredictiveAgent trails badly at 6.4% at 270.2 avg cost. Q-Learning's off-policy nature (learning the optimal policy regardless of the exploratory actions taken) gives it a slight edge over SARSA's onpolicy updates. PredictiveAgent's very high average cost confirms it's spending many steps without

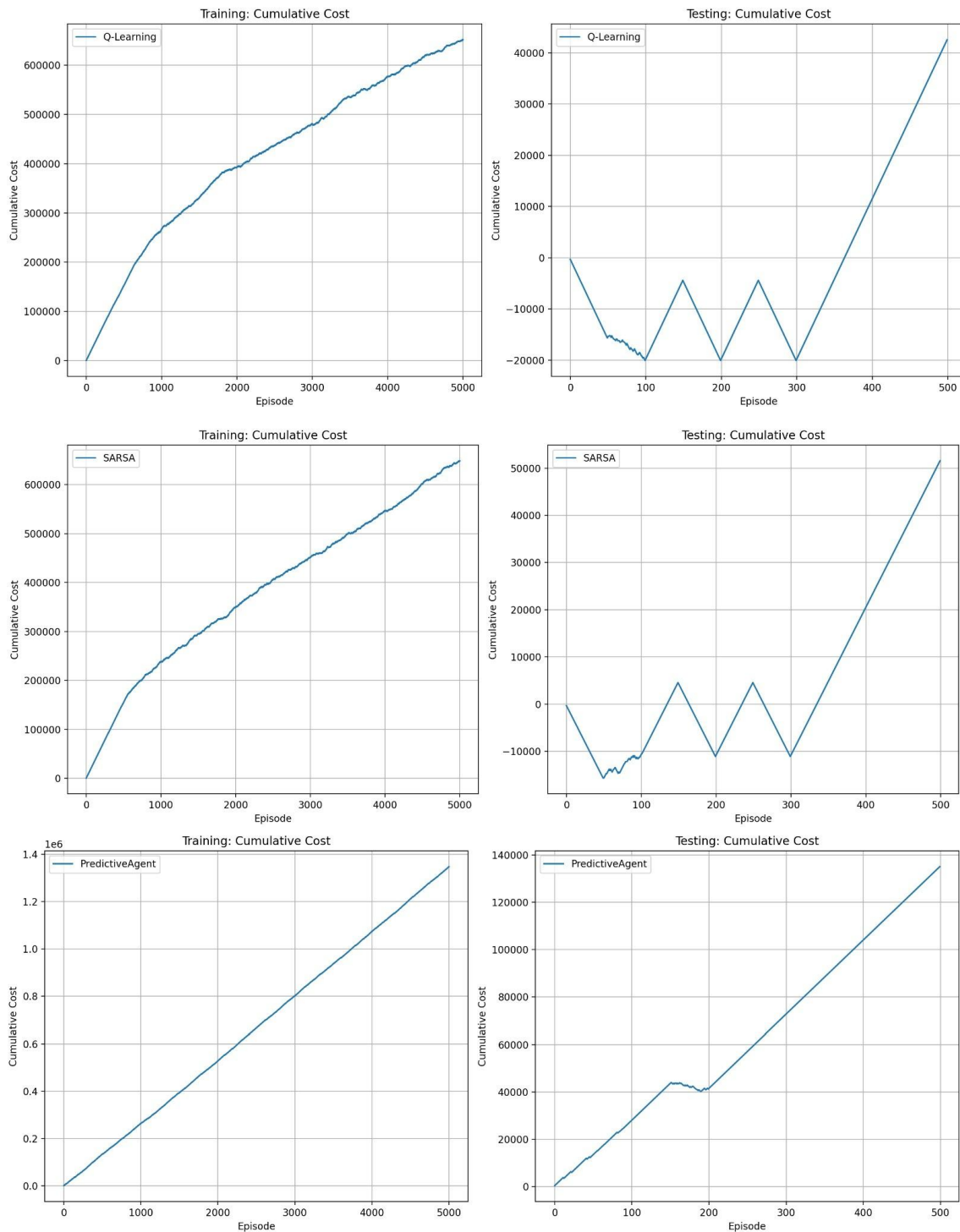
reaching the goal. Notably, SARSA has fewer avg steps (674 vs 755 for Q-Learning), suggesting it finds shorter paths but less reliably wins — possibly being more conservative due to on-policy learning.

Policy Differences and Convergence

The PredictiveAgent applies a naive rule-based strategy without any state-based learning, which explains its consistently high average cost (270.2) and low win rate (6.4%). It serves as a useful baseline that highlights the value of learning from interaction.

Between the two RL methods, Q-Learning (off-policy) learns more aggressively by always bootstrapping from the greedy action, leading to deeper cumulative cost reductions during training (reaching approximately -20,000) and stronger test-time performance with the highest win rate (36.8%) and lowest average cost (85.1). SARSA (on-policy) is more conservative since it updates based on the actions actually taken during exploration, achieving smaller cumulative cost reductions (~15,000) but still competitive results, with a 33.0% win rate and shorter average episodes (674 vs 755 steps).

Overall, Q-Learning is the stronger performer based on all available metrics, while SARSA offers a more stable and cautious alternative that still significantly outperforms the rule-based baseline.



1.2.1

Training phase

The training curve shows very strong robustness. The 5 individual seed runs (gray lines) stay tightly clustered around the mean throughout all 2,500 episodes, and the standard deviation band (light blue shading) remains narrow relative to the scale of cumulative cost (~400k by the end). This tight grouping means the Qlearning algorithm converges reliably regardless of random initialization — the policy learned is not sensitive to the seed. The slight fanning out toward the end (episodes 1,500–2,500) is completely

normal: variance in cumulative metrics naturally compounds over time, so even small per-step differences grow into visible gaps by episode 2,500. Overall, training robustness is high.

Testing phase

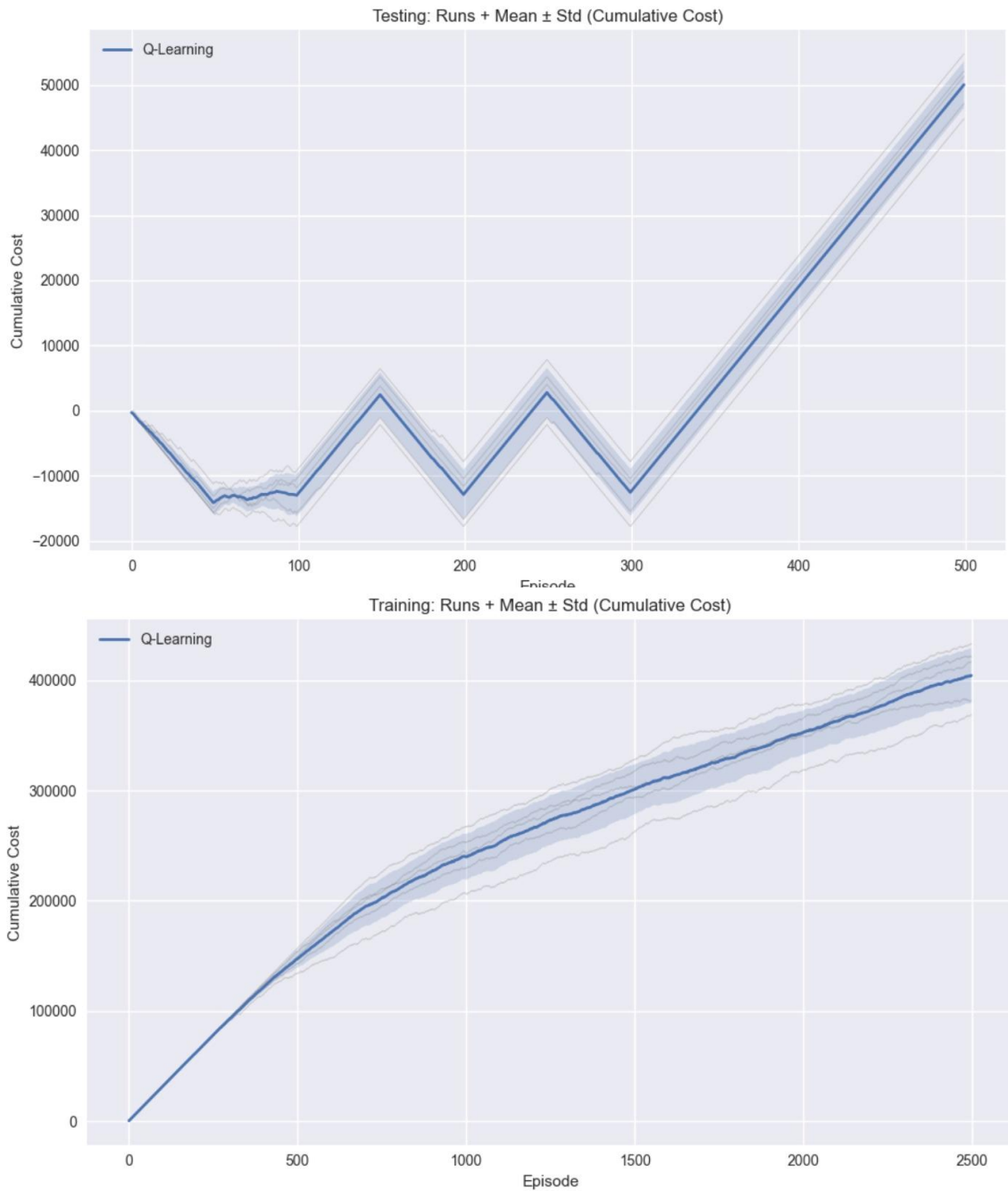
The testing results are equally robust in terms of seed consistency. All 5 seed runs remain very tightly clustered around the mean throughout the entire 500 episodes, including the later phase where cumulative cost rises sharply. The standard deviation band stays narrow from start to finish, meaning the learned policy generalizes in a highly reproducible and deterministic way across seeds.

What the testing graph actually highlights is not a variance problem, but a systematic behavioral pattern shared by all runs. The zigzag structure — dips around episodes ~50, ~200, and ~300 followed by recoveries — is reproduced identically across all seeds, confirming it reflects a real periodic structure in the test environment rather than noise. After episode ~300, all seeds consistently accumulate cost at a much higher rate, which points to a structural limitation of the learned Q-table policy when facing longer test horizons — not a robustness issue.

Overall conclusion

Both training and testing demonstrate high robustness: the algorithm is stable and reproducible across all 5 seeds. The main concern is not consistency across runs, but rather the policy's tendency to accumulate increasing cost during testing over longer horizons, which suggests the Q-table may not generalize perfectly beyond the conditions seen during training.

1.2.2



No the final policy is not optimal. The graph shows three clear signs:

1. Cumulative cost grows unboundedly ($\sim 50,000$ by episode 500) — an optimal policy would converge to a stable value, not keep rising.
2. No convergence — the slope actually increases after episode 350, the opposite of what learning should produce.
3. High variance across runs — the wide spread of gray lines means the agent behaves inconsistently, indicating it never found a reliable strategy.

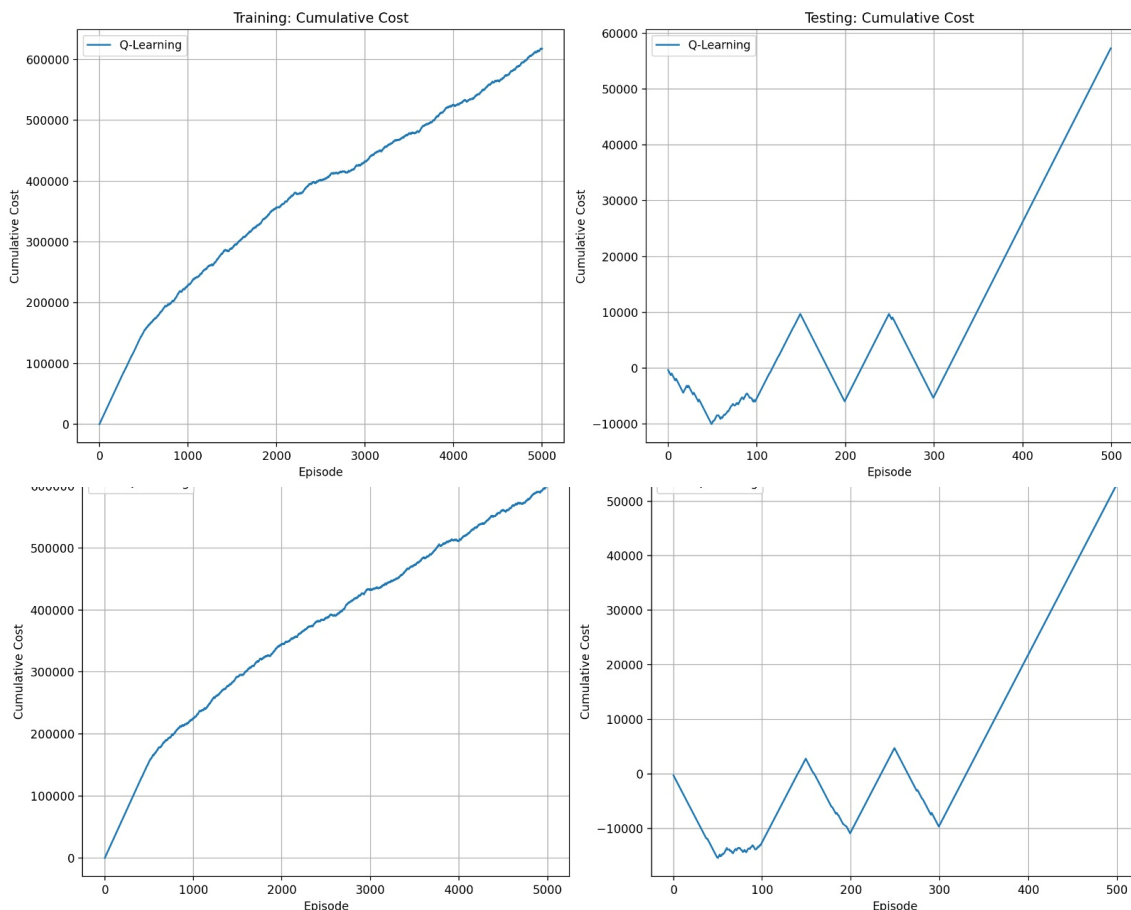
The policy fails on harder cases because Q-Learning's table-based approach can't handle a large or complex state space.

1.2.3

Epsilon (ϵ) controls the exploration-exploitation trade-off. In the original configuration, a higher epsilon means the agent explores more randomly. When epsilon is **increased**, the agent explores more aggressively early on, which can help discover better policies but slows convergence — visible as a steeper or noisier training cumulative cost curve. When epsilon is **decreased**, the agent exploits its current knowledge sooner, potentially converging faster but risking getting stuck in suboptimal policies.

Epsilon_decay controls how quickly exploration reduces over episodes. A **faster decay** (smaller multiplier) causes the agent to shift to exploitation sooner, which can lead to lower training costs in later episodes but may underfit if the Qtable hasn't been sufficiently populated. A **slower decay** keeps exploration active longer, which generally smooths the training curve but may delay convergence.

Comparing the two images, both runs show very similar training cumulative cost curves (reaching ~600k by episode 5000) and similar oscillatory testing behavior before a sharp rise after episode 300–400. This suggests the exploration parameter differences between the two runs were modest and the agent's overall learning trajectory was largely unaffected — implying the environment may be robust to small epsilon changes, or that the Q-table discretization is the dominant bottleneck.



1.2.4

When the agent is trained exclusively on one fish type (e.g., Sturgeon), the Q-table learns state-action values **specific to that fish's behavior** (speed, size, movement patterns). If then tested on other fish types, the learned policy is likely to **generalize poorly**, since the state transitions and reward structure differ.

Compared to the original multi-fish training, you would expect **lower training costs** (the agent specializes efficiently) but **higher test cumulative costs and more avg. steps** on unseen fish types, as the agent applies an ill-fitted policy. The sharp cost increase seen in the testing plots after episode ~350 in both images could reflect exactly this kind of distribution mismatch — the agent fails to adapt when encountering unfamiliar dynamics it was never trained on.

1.3 the indexes were in amounts of 50 and the change we did was to make it in amounts of 20, making the program more accurate over a longer time but harder to train since it has now much more parameters

we can see that in the graphics since it doesn't seem to be near convergence and still the normal one gave a win rate of 37 in testing and our gave 35

